Document number: P0710R1
Date: 2017-07-13
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2014
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "ready" Issues for the July, 2017 (Toronto) meeting

Section references in this document reflect the section numbering of document WG21 N4659.

2253. Unnamed bit-fields and zero-initialization

Section: 12.2.4 [class.bit] **Status:** ready **Submitter:** Aaron Ballman **Date:** 2016-03-23

The current wording of the Standard does not clearly state that zero-initialization applies to unnamed bit-fields.

Notes from the December, 2016 teleconference:

The consensus was that unnamed bit-fields do constitute padding; more generally, padding should be normatively defined, along the lines suggested in 12.2.4 [class.bit] paragraphs 1-2.

Proposed resolution (March, 2017):

1. Change 6.9 [basic.types] paragraph 4 as follows:

> The *object representation* of an object of type `T` is the sequence of *N* `unsigned char` objects taken up by the object of type `T`, where *N* equals `sizeof(T)`. The *value representation* of an object is the set of bits that hold the value of type `T`. **Bits in the object representation that are not part of the value representation are *padding bits*.** For trivially copyable types, the value representation is a set of bits in the object representation that determines a *value*, which is one discrete element of an implementation-defined set of values.[44]

2. Change 11.6 [dcl.init] paragraph 6 as follows:

> To *zero-initialize* an object or reference of type `T` means:
>
> - if `T` is a scalar type (6.9 [basic.types]), the object is initialized to the value obtained by converting the integer literal `0` (zero) to `T`;[103]
> - if `T` is a (possibly cv-qualified) non-union class type, **its padding bits are initialized to zero bits and** each non-static data member and each base-class subobject is zero-initialized ~~and padding is initialized to zero bits~~;
> - if `T` is a (possibly cv-qualified) union type, **its padding bits are initialized to zero bits and** the object's first non-static named data member is zero-initialized ~~and padding is initialized to zero bits~~;

- ...

3. Change 12.2.4 [class.bit] paragraph 1 as follows:

> ...The *constant-expression* shall be an integral constant expression with a value greater than or equal to zero. The value of the integral constant expression may be larger than the number of bits in the object representation (6.9 [basic.types]) of the bit-field's type; in such cases the extra bits are ~~used as~~ padding bits **(6.9 [basic.types])**~~and do not participate in the value representation (6.9 [basic.types]) of the bit-field~~. Allocation of bit-fields...

4. Change 6.9.1 [basic.fundamental] paragraph 1 as follows:

> ...For narrow character types, all bits of the object representation participate in the value representation. [*Note:* A bit-field of narrow character type whose length is larger than the number of bits in the object representation of that type has padding bits; see ~~12.2.4 [class.bit]~~ **6.9 [basic.types]**. —*end note*] For unsigned narrow character types...